RPG IV Mise en œuvre

Format libre

Problématique

- Moderniser le langage
  - Dans sa syntaxe
  - Dans ses possibilités
- Standardiser le langage
  - Le rendre plus accessible
  - Améliorer la communication entre développeurs RPG et autres

Principe

- Depuis la V5R1 du System i, le format libre est disponible pour les spécifications de traitement uniquement (carte C)
- Le format libre permet de s’affranchir du colonage
- Souvent appelé RPG V
- Ce n’est pas un nouveau langage, mais une nouvelle syntaxe
  - Les code opérations sont les mêmes

RPG IV : les syntaxes

- Fixe
  - Syntaxe historique
  - Peu lisible
    - Les structures n'apparaissent pas
  - La taille des différentes zones est limitée
    - Problématique pour les tableaux et les DS qualifiées
- Libre
  - Syntaxe plus moderne
    - Syntaxe dite naturelle
  - Syntaxe proche de la plupart des langages (Java, C, PHP …)
  - Permet l'indentation
    - Lisibilité du code
  - Permet d'avoir plus de place disponible sur la ligne

Règle syntaxique

- Un bloc de code en format libre doit
  - Commencer par /free
  - Se terminer par /end-free
  - /free et /end-free sont des instructions de pré-compilation
    - Commencent en colonne 7
    - Indique au compilateur la syntaxe utilisée

Règle syntaxique

- Un bloc comporte des instructions
  - Une instruction peut être écrite entre les colonnes 8 et 80
  - Une instruction se termine par ;
  - Une seule instruction par ligne
  - Une instruction peut être multi-lignes sans symbole de continuation
  - Règle syntaxique générale :

    Code opération facteur1 facteur2 résultat ;
- Commentaire
  - Commence par //

    A partir de la colonne 8

    Tout ce qui est après // est un commentaire

Règles syntaxiques – Exemple

```
C     cle           chain     fichier
 /free
  chain cle fichier ;
 /end-free


C* commentaire en format fixe
 /free
  // commentaire en format libre
  chain cle fichier ;  // le commentaire peut être ici

  * ce commentaire est incorrect
 C* ceci est incorrect aussi bien pour la carte C que pour le commentaire
 /end-free


 /free
  chain cle        // instruction sur plusieurs lignes
        fichier ;  // le ; indique la fin de l'instruction
 /end-free
```

Indentation – Exemple

- L'indentation augmente la lisibilité en mettant en évidence les structures de contrôle

```
/free
 chain(e) cle fichier ;
 select ;
    when %error ;
       dsply 'Erreur' ;
    when %found ;
       if cle <> 0
          dsply 'Ok' ;
       endif ;
    endsl ;
 endsl ;
/end-free
```

Contraintes

- Les indicateurs ne sont plus utilisables
  - Pour conditionner une ligne
  - En résultat d’instruction
- Certains codes opérations ne sont pas autorisés en format libre
  - Les codes opérations obsolètes
  - Les codes opérations ayant un équivalent plus moderne, en fonction intégrée pour la plupart

Remplacement des indicateurs

- Indicateur de conditionnement
  - Colonné

    ```
    C   01cle          chain     fichier
    ```

  - Libre

    ```
    if *in01 = *on ;
            chain cle fichier ;
    endif ;
    ```

- Indicateur résultant
  - Pour les opérations sur les fichiers

    Utiliser les fonctions intégrées %eof, %error, %found, %equal, %status et %open

  - Pour les autres codes opérations

    Des fonctions intégrées au cas par cas

Codes opération supprimés (1/3)

Codes	Commentaire ou remplacement
ADD, SUB, DIV, MULT, Z-ADD, Z-SUB, SQRT, MVR	Remplacés par les opérateurs naturels : + - * / Et les fonctions intégrées %div, %rem, %sqrt
EXTRCT	%subdt
ADDDUR, SUBDUR	Opérateurs + et – Et les fonctions intégrées de durée (%years, %days …)
ALLOC, REALLOC	%alloc, %realloc
BITON, BITOFF	%bitand, %bitnot, %bitor
SETON, SETOFF	EVAL
CABxx, CASxx	Obsolète. Utilisez IF ou SELECT
CHECK, CHECKR	%check, %checkr
DOUxx, DOWxx	DOU et DOW
IFxx, ANDxx, Orxx	Utilisez IF, AND et OR
CALL, CALLB	CALLP avec déclaration

Codes opération supprimés (2/3)

Codes	Commentaire ou remplacement
CAT	Opérateur +
COMP	Opérateur < > = <= >= <>
DEFINE	Obsolète. Equivalent en carte D (déclaration)
DIV	Opérateur / ou %div
DO	FOR
GOTO	Obsolète
KLIST, KFLD	%kds
LOOKUP	%lookup, %tlookup
MHHZO, MHLZO, MLHZO, MLLZO	%bitand, %bitnot, %bitor
MOVE, MOVEL, MOVER	EVAL, EVALR Pour les conversions : %char, %date, %dec, %dech …

Codes opération supprimés (3/3)

Codes	Commentaire ou remplacement
MOVEA	%subarr
OCCUR	%occur
PLIST / PARM	Remplacé par prototype en carte D (déclaration)
SCAN	%scan
SUBST	%subst
TAG	Obsolète
TESTB	%bitand
TESTN	Obsolète
TESTZ	Obsolète
TIME	%time, %date, %timestamp
XFOOT	%xfoot
XLATE	%xlate

Equivalence directe - Exemple

```
 * Format fixe

d dateEur         s               d   inz datfmt( *eur )
d jour            s              2s 0 inz
d mois            s              2s 0 inz

 * Alimentation de la date
c     *usa          move      *date         dateEur
 * Extraction jour et mois
c                   extrct    dateEur:*d    jour
c                   extrct    dateEur:*m    mois
  ...
c                   seton                                         lr


  // format libre

 /free

  // Alimentation de la date
  dateEur = %date() ;
  // extraction jour et mois
  jour = %subdt( dateEur : *d ) ; // ou %subdt( dateEur : *days )
  mois = %subdt( dateEur : *m ) ; // ou %subdt( dateEur : *months )
  ...
  *inlr = *on ;

 /end-free
```

MOVE - Conversions

- L’instruction MOVE effectue des conversions implicites
- En format libre, IBM demande à ce que les conversions soient explicites via l’utilisation des fonctions intégrées
  - %char
  - %date
  - %unt, %int
  - %dec
  - …

MOVE - Conversions - Exemple

```
 * Variable de type date
d Date            s               d   datfmt( *eur )
 * Variable numérique pour date en *YMD
d Num6            s              6s 0 inz( 100521 )
 * Variable numérique pour date en *iso
d Num8            s              8s 0
 * Variable alpha qui représente la date 2010-05-21
d Char6           s              6a   inz( '100521' )
 * Variable alpha pour date en *eur
d Char10          s             10a

 * Conversion de numérique en date
c     *YMD          move      Num6          Date
 * Conversion de date en numérique
c     *ISO          move      Date          Num8
 * Conversion de date en alpha
c     *EUR          move      Date          Char10
 * conversion alpha vers date
c     *YMD0         move      Char6         Date

  // Conversion de numérique en date
  Date = %date( Num6 : *YMD ) ;
  // Conversion de date en numérique
  Num8 = %dec( Date : *ISO ) ;
  // Conversion de date en alpha
  Char10 = %char( Date : *EUR ) ;
  // conversion alpha vers date
  Date = %date( Char6 : *YMD0 ) ;
```

MOVE - Conversions - Exemple

```
 * Variable numérique pour date en *YMD
d Num6            s              6s 0 inz( 100521 )
 * Variable alpha qui représente la date 2010-05-21
d Char6           s              6a   inz( '100521' )
 * Variable alpha pour date en *eur
d Char10          s             10a
 * Variable de type packé 6,2
d Pack6_2         s              6p 2 inz( 123,45 )

 * conversion alpha vers numérique entier
c                   move      Char6         Num6
 * conversion numérique vers alpha
c                   move      *blank        Char10
c                   move      Pack6_2       Char10
 * conversion alpha vers numérique
c                   movel(p)  '    987654'  Char10
c                   move      Char10        Pack6_2

  // conversion alpha vers numérique entier
  Num6 = %int( Char6 ) ;
  // conversion numérique vers alpha
  Char10 = %editc( Pack6_2 : '1' ) ;  // avec code édition
  Char10 = %char( Pack6_2 ) ;         // sans code édition
  // conversion alpha vers numérique
  Pack6_2 = %dec( Char10 : 6 : 2 ) ;
```

KLIST/KFLD - Clés

- Le code instruction KLIST suivi de KFLD permet de définir une clé composite pour l'accès à un fichier
  - Représentée par un nom
  - Formée de sous-zones
- Il peut exister plusieurs KLIST pour le même fichier (clés partielles)
- En format libre, nous utiliserons
  - Une DS externe au format de la clé du fichier
    - Possède un nom
    - Formée de sous-zones
  - Une fonction intégrée %KDS permettant d'utiliser une DS en tant que clé (partielle ou non)

KFLD/KLST - Exemple

```
 * fichier des clients par code postal. Clé composée : CPLCLI + NUMCLI
fclientsi1 uf a e           k disk

d kcpcli          s                   like( CPLCLI )  inz
d knumcli         s                   like( NUMCLI )  inz

c     KCLIENTS      KLIST
c                   KFLD                    kcpcli
 * Lecture de tous les clients de Paris 3ème
c                   move      '75003'       kcpcli
c     KCLIENTS      setll     clientsi1
c     KCLIENTS      reade     clientsi1
c                   dow       not %eof
 *                  ... traitement
c     KCLIENTS      reade     clientsi1
c                   enddo

  // Format libre
d CleClients    e ds                  extname( CLIENTSI1 : *key )
d                                     qualified
d                                     inz

  CleClients.CPLCLI = '75003' ;
  setll %kds( CleClients : 1 ) clientsi1 ;
  reade %kds( CleClients : 1 ) clientsi1 ;
  dow not %eof ;
     // ... traitement
     reade %kds( CleClients : 1 ) clientsi1 ;
  enddo ;
```

CALL – Appel de programme

- Les instructions d'appel CALL et CALLB (appel lié) ne sont plus autorisées, au profit de l'instruction CALLP (appel prototypé)
  - Les instructions PLIST et PARM ne sont pas supportées
- Il faut définir un prototype pour les programmes que l'on appelle
  - Le mot-clé EXTPGM permet d'indiquer que le prototype décrit un programme et non une procédure
    - Pas de valeur de retour possible
    - Les paramètres sont transmis par référence
    - L'appel reste dynamique

CALL – Exemple

```
 * Format fixe

d Parm1           s             10a
d Parm2           s               d
d Parm3           s             15p 5

d NomPgm          s             10a   inz( 'AUTRE_PGM' )

c                   call      'CALC_JOUR'
c                   parm                    Parm1
c                   parm                    Parm2
c                   parm                    Parm3

c                   call      NomPgm

 * Format libre

d Cacl_Jour       pr                  extpgm( 'CALC_JOUR' )
d  P1                           10a
d  P2                             d
d  P3                           15p 5

d PgmGenerique    pr                  extpgm( NomPgm )


  callp Calc_Jour( parm1 : parm2 : parm3 ) ;   // appel du programme CALC_JOUR

  callp PgmGenerique() ;                       // appel du programme en fonction de NomPgm
```

Synthèse

- Le format libre représente un véritable confort
  - Dans la syntaxe
    - Indentation
    - Imbrication
  - De simplification
    - L'imbrication permet de diminuer le nombre de variables intermédiaires
- Mais inachevée
  - Uniquement les cartes C
    - Et encore, entre les colonnes 7 et 80 …
  - Depuis la V7R1 TR7, le RPG IV est totalement en format libre !
    - Regardez le format libre pour les autres spécifications